

Discrete Mathematics in Practice: A Formal Analysis of an AI-Powered Skin Disease Diagnosis System

Course: Discrete Mathematics

Institution: MSA University

Author: Yousef Yasser

Date: April 2026

Table of Contents

1. Abstract
 2. Introduction
 3. System Overview
 4. Concept I — Set Theory
 5. Concept II — Relations and Functions
 6. Concept III — Graph Theory
 7. Concept IV — Propositional and Predicate Logic
 8. Concept V — Combinatorics
 9. Concept VI — Algorithms and Complexity
 10. Mathematical Modeling: Unified Formal Description
 11. Implementation Mapping
 12. Diagram Placeholders
 13. Conclusion
 14. References
-

1. Abstract

This project applies the core concepts of Discrete Mathematics to a real, production-grade system: a mobile application that diagnoses skin diseases from photographs using Artificial Intelligence. The system employs a three-stage pipeline consisting of a Variational Autoencoder (VAE) for anomaly gating, an EfficientNet ensemble (B2 and B3) for classification, and Score-CAM for explainability heatmap generation — all running entirely offline on a Flutter mobile application. Rather than treating Discrete Mathematics as an abstract discipline, this paper demonstrates how Set Theory, Relations and Functions, Graph Theory, Logic, Combinatorics, and Algorithm Analysis are not merely applicable but are in fact the precise formal language that describes every decision, structure, and

computation in this system. Each mathematical concept is introduced, formally defined, and then directly mapped to a concrete component or decision within the project, showing that the mathematics is not added on top of the engineering — it is the engineering, expressed rigorously.

2. Introduction

Discrete Mathematics occupies a foundational role in computer science and software engineering. Unlike continuous mathematics, which describes quantities that change smoothly, discrete mathematics concerns itself with structures that are countable, finite, or logically structured — graphs, sets, logical statements, sequences, and algorithms. These are precisely the kinds of objects that appear in every software system ever built.

The system analyzed in this paper is a dermatology screening application designed as a graduation project at MSA University. Its purpose is to assist both patients and healthcare providers in identifying three common skin conditions — Acne, Eczema, and Tinea — from a photograph taken on a mobile phone. The application runs entirely offline, meaning all inference happens on the device itself without any network connection. This is a non-trivial engineering constraint that forces every component of the system to be efficient, deterministic, and formally correct.

The pipeline can be described at a high level as follows. A user opens the application, selects a body part, captures or uploads a photograph, and the application processes that image through three sequential AI models before displaying a diagnosis alongside a visual explanation. Beneath this seemingly simple interaction lies a sophisticated chain of mathematical operations: image decomposition into overlapping patches, threshold-based logical gating, probabilistic classification via a neural network ensemble, weighted channel selection for explainability, and finally storage and retrieval of results in a relational database.

The goal of this paper is to show, with precision and depth, exactly how six branches of Discrete Mathematics — Set Theory, Relations and Functions, Graph Theory, Logic, Combinatorics, and Algorithm Analysis — manifest in this system. The analysis covers the training pipeline (Python, run on Kaggle GPUs), the mobile application (Flutter/Dart), and the AI inference pipeline (TensorFlow Lite).

3. System Overview

This section establishes a clear picture of the system's architecture and data flow before the formal mathematical analysis begins.

Training Phase (Offline, Kaggle)

The AI models were trained over twelve iterative phases on Kaggle using GPU acceleration. The training began with MobileNetV2 as a baseline and progressively advanced through EfficientNetB0, B2, B3, and B4 architectures. Techniques such as focal loss, label smoothing, test-time augmentation (TTA), and ensemble averaging were applied to push classification accuracy toward 93%. The final deployed models are EfficientNetB2 and EfficientNetB3, converted to TensorFlow Lite format.

Inference Phase (On-Device, Flutter)

When the user submits an image, the application executes the following pipeline:

1. **Preprocessing:** If the image exceeds 1280 pixels in either dimension, it is downsampled using linear interpolation.
2. **VAE Anomaly Gate:** A Variational Autoencoder processes the image in overlapping 64×64 pixel patches using a sliding window with stride 32. For each patch, the model computes a reconstruction error (Mean Squared Error). If fewer than 20% of patches exceed the anomaly threshold, the image is classified as "Normal" and the pipeline terminates early.
3. **CNN Ensemble Classification:** The image is passed through both EfficientNetB2 (resized to 260×260) and EfficientNetB3 (resized to 300×300). The softmax output vectors from both models are averaged, and the class with the highest averaged probability is selected as the diagnosis.
4. **Score-CAM Heatmap:** The top 30 feature map channels from the EfficientNetB3 backbone are extracted. Each channel is used to mask the input image, and the masked image is re-fed through the network to measure the change in class score. Channels are assigned weights via softmax normalization. The final heatmap is a weighted sum of the 30 channel activations, blended with the original image using a 60/40 ratio.
5. **Storage:** The diagnosis, confidence score, class probabilities, heatmap file path, body part, and timestamp are stored in a local SQLite database.

Application Structure

The Flutter application supports two user roles — Patient and Healthcare Provider (Specialist). The navigation system uses a directed routing graph managed by `go_router`. State is managed through Riverpod providers, and all data persists locally via SQLite.

4. Concept I — Set Theory

4.1 Formal Definition

A **set** is a well-defined, unordered collection of distinct elements. Sets are denoted by capital letters and their elements are listed within curly braces. The fundamental operations on sets include union ($\cup$), intersection ($\cap$), difference ($-$), and complement (A^c). A set A is a **subset** of B , written $A \subseteq B$, if every element of A is also in B .

The **power set** of A , written $P(A)$, is the set of all subsets of A . For a set with n elements, $|P(A)| = 2^n$.

A **partition** of a set S is a collection of non-empty, mutually disjoint subsets of S whose union equals S . Formally, $\{A_1, A_2, \dots, A_k\}$ is a partition of S if: - $A_i \neq \emptyset$ for all i - $A_i \cap A_j = \emptyset$ for all $i \neq j$ - $A_1 \cup A_2 \cup \dots \cup A_k = S$

4.2 Application to the System

The Diagnosis Class Set

The most immediate application of Set Theory is the definition of the output space. The system classifies every image into one of four possible outcomes. We define the complete output set as:

```
D = {Acne, Eczema, Tinea, Normal}
```

The subset **D_disease** = {Acne, Eczema, Tinea} $\subset$ **D** represents only the pathological conditions, which are the targets of the CNN ensemble. The element "Normal" is only returned by the VAE gate and is never produced by the CNN.

Dataset Partitioning

Before training, the available labeled images must be split into three disjoint subsets. Let U be the universal set of all available images. The partition $\{\text{Train}, \text{Val}, \text{Test}\}$ satisfies:

```
Train  $\cap$  Val =  $\emptyset$ 
Train  $\cap$  Test =  $\emptyset$ 
Val  $\cap$  Test =  $\emptyset$ 
Train  $\cup$  Val  $\cup$  Test =  $U$ 
```

This is not merely a coding convention — it is a formal requirement for valid model evaluation. Any overlap between Train and Test violates this partition, which would cause the model's reported accuracy to be artificially inflated. The system uses an approximately 90/5/5 stratified split, meaning class proportions are preserved within each subset.

Feature Channel Selection in Score-CAM

The EfficientNetB3 backbone produces n feature map channels at its final convolutional layer. Score-CAM operates by selecting a subset of these channels. Let:

```
 $C = \{c_1, c_2, \dots, c_n\}$  be the complete set of feature channels
 $S \subseteq C$  with  $|S| = 30$  be the selected top-K subset
```

The selection criterion is the L2 norm (magnitude) of each channel's activation. The subset S is the 30 highest-magnitude channels. The complement $S_{\text{discarded}} = C - S$ represents channels whose contribution to the final heatmap is considered negligible. This subset selection is an applied instance of the concept of choosing a representative subset from a larger set based on a ranking function.

Patch Set in the VAE Sliding Window

For an input image of height H and width W , the sliding window generates a set of patches:

$$P = \{ p_{ij} \mid 0 \leq i \leq \lfloor (H-64)/32 \rfloor, 0 \leq j \leq \lfloor (W-64)/32 \rfloor \}$$

Each patch p_{ij} is a 64×64 pixel sub-image extracted at position $(32i, 32j)$. The patches overlap (since stride $32 <$ window size 64), meaning P is not a partition of the image. The subset of anomalous patches is:

$$P_{\text{anomalous}} = \{ p_{ij} \in P \mid \text{MSE}(p_{ij}) > \theta \}$$

where θ is the reconstruction error threshold. The decision to classify or return "Normal" depends on $|P_{\text{anomalous}}| / |P|$.

5. Concept II — Relations and Functions

5.1 Formal Definition

A **relation** R from set A to set B is a subset of the Cartesian product $A \times B$. We write aRb or $(a, b) \in R$ to mean that element a is related to element b .

A **function** $f: A \rightarrow B$ is a special relation in which every element of A is related to exactly one element of B . A is called the domain and B the codomain. A function is: - **Injective (one-to-one)** if $f(a_1) = f(a_2)$ implies $a_1 = a_2$ - **Surjective (onto)** if for every $b \in B$ there exists an $a \in A$ with $f(a) = b$ - **Bijective** if it is both injective and surjective

Function composition is defined as $(g \circ f)(x) = g(f(x))$, where the output of f is the input to g .

5.2 Application to the System

The Classification Function

The core operation of the CNN ensemble is a function that maps an image to a diagnosis with associated confidence:

$$f: \text{ImageSpace} \rightarrow D_{\text{disease}} \times [0, 1]$$

where ImageSpace is the set of all valid RGB images and $[0, 1]$ is the confidence interval. This function is not injective (many different images may produce the same diagnosis) and whether it is surjective depends on whether all three diseases can actually be detected (they can, by design). This is a many-to-few function, and its formal properties matter: if the function were not well-defined (i.e., some images could map to two diagnoses simultaneously), the system would be logically broken.

The Preprocessing Function

Image preprocessing is a function:

```
preprocess: ImageSpace  $\rightarrow$  ImageSpace224
preprocess(img) = downsample(img) if max(H, W) > 1280, else img
```

This is a conditional identity function — it behaves as the identity on images already within bounds, and as a downsampling function otherwise. It is a total function (defined for all inputs) and produces images suitable for the models.

The Ensemble Averaging Function

Let $f_{B2}: \text{ImageSpace} \rightarrow \mathbb{R}^3$ and $f_{B3}: \text{ImageSpace} \rightarrow \mathbb{R}^3$ be the softmax output functions of EfficientNetB2 and B3 respectively. Each maps an image to a probability vector over the three disease classes. The ensemble function is:

```
ensemble(x) = (f_B2(resize260(x)) + f_B3(resize300(x))) / 2
```

This is a function of type $\text{ImageSpace} \rightarrow \mathbb{R}^3$ constructed by composing each model with its corresponding resize function, then averaging. The final diagnosis is:

```
diagnosis(x) = argmax(ensemble(x))
```

The argmax operation is a function $\text{argmax}: \mathbb{R}^3 \rightarrow \{0, 1, 2\}$, mapping the probability vector to the index of the largest element.

The User-Scan Relation

In the SQLite database, the relationship between users and scan records forms a binary relation:

```
R_user_scan  $\subseteq$  Users  $\times$  Scans
```

This is a one-to-many relation: a single user may have many scan records, but each scan record belongs to exactly one user. This is the mathematical foundation of the foreign key constraint in the database schema.

The Score-CAM Weighting Function

Score-CAM assigns a weight to each selected channel $c_i \in S$. The scoring process feeds masked versions of the image through the network and measures the change in the target class score. Let $\text{score}(c_i)$ denote this scalar score. The weighting function is:

```
w: S  $\rightarrow$  [0, 1]
w(ci) = exp(score(ci)) /  $\sum_j$  exp(score(cj))
```

This is the softmax function applied over the selected channel set S . It is a well-defined function (total, positive-valued, outputs sum to 1) and transforms raw scores into a proper probability distribution over channels — formally a probability mass function.

6. Concept III — Graph Theory

6.1 Formal Definition

A **graph** $G = (V, E)$ consists of a set of **vertices** V and a set of **edges** E , where each edge connects a pair of vertices. Graphs can be: - **Undirected**: edges have no direction, $E \subseteq \{\{u, v\} \mid u, v \in V\}$ - **Directed (digraph)**: edges have direction, $E \subseteq V \times V$

A **path** in a graph is a sequence of vertices where each consecutive pair is connected by an edge. A **cycle** is a path that starts and ends at the same vertex. A graph with no cycles is called a **DAG** (Directed Acyclic Graph). A **tree** is a connected, acyclic undirected graph.

6.2 Application to the System

The Application Navigation Graph

The Flutter application's screen routing system is a directed graph. Define:

```
V_screens = { Welcome, Login, RoleSelection, PersonalInfo, AllSet,
HowItWorks, Home, SpecialistDashboard, BodyPartSelection, Capture,
Analyzing, DiagnosisResult, History, Profile, EditProfile,
Notifications, HelpSupport, ForgotPassword }
```

Each directed edge $(u, v) \in E$ represents a valid navigation transition from screen u to screen v . For example:

```
(Welcome, Login), (Login, RoleSelection), (RoleSelection, PersonalInfo)
(Home, BodyPartSelection), (BodyPartSelection, Capture), (Capture,
Analyzing)
(Analyzing, DiagnosisResult), (DiagnosisResult, History)
```

This navigation graph is largely a DAG — the onboarding flow is strictly linear and acyclic. The main application section contains cycles (the user can return to the home screen from most screens), making the overall graph a general directed graph. The `go_router` library in Flutter manages this graph as a routing tree, ensuring that every screen is reachable from exactly one defined set of entry points.

The in-degree and out-degree of vertices are meaningful: the Home screen has high in-degree (many screens return to it) and high out-degree (it leads to many features). The DiagnosisResult screen has out-degree ≥ 2 (the user can navigate to History or return Home), reflecting a fork in the navigation graph.

The Pipeline as a Directed Acyclic Graph

The AI inference pipeline is a directed acyclic graph of computation stages:

```
V_pipeline = { Input, Preprocess, VAEGate, CNNEsemble, ArgmaxClassify,
ScoreCAM, Store, Display }
```

The directed edges are:

```
(Input → Preprocess), (Preprocess → VAEGate)
(VAEGate → CNNEnsemble) [if anomalous]
(VAEGate → Display) [if normal – early exit]
(CNNEnsemble → ArgmaxClassify), (ArgmaxClassify → ScoreCAM)
(ScoreCAM → Store), (Store → Display)
```

This is a DAG with a branch at the VAEGate node. The two outgoing edges from VAEGate represent a conditional fork — the pipeline takes different paths depending on the anomaly ratio. The absence of cycles is guaranteed: the pipeline never re-feeds its output into an earlier stage. This acyclicity is crucial for termination — a cyclic pipeline could run indefinitely.

The Patch Lattice Graph

The sliding window over the image generates a grid of patches where adjacent patches (those whose windows overlap) can be considered connected. Define a graph $G_{\text{patches}} = (P, E_{\text{overlap}})$ where:

```
 $(p_{ij}, p_{kl}) \in E_{\text{overlap}}$  iff  $|i-k| \leq 1$  and  $|j-l| \leq 1$ 
```

This forms a grid graph (also called a lattice graph). The anomalous patches $P_{\text{anomalous}}$ form a subgraph. If $P_{\text{anomalous}}$ is connected (all anomalous patches form a single connected component), it suggests a localized lesion. If it is disconnected, the lesion may be diffuse or the image may contain noise. This connectivity analysis could, in principle, be used to localize the disease region spatially — an extension beyond the current system but grounded in graph theory.

The Model Training Dependency Graph

The twelve training phases form a directed graph of dependencies:

```
Phase 1 (MobileNetV2 baseline) → Phase 2 (fine-tuning) → Phase 3 (anti-
overfitting)
Phase 3 → Phase 4 (deeper unfreezing)
Phase 5 (EfficientNetB0) → independent branch
Phase 7 (EfficientNetB2) → Phase 8 (B0+B2 ensemble) → Phase 9 (dual B2)
Phase 10 (EfficientNetB3) → Phase 11 (TTA sweep) → Phase 12
(EfficientNetB4)
```

Each node represents a trained model. Each edge $(A \rightarrow B)$ means that model B was initialized from the weights of model A or that B's design decisions were informed by A's evaluation results. This is a DAG — no model is a dependency of its own ancestor.

7. Concept IV — Propositional and Predicate Logic

7.1 Formal Definition

Propositional Logic deals with statements that are either true or false. Logical connectives include conjunction ($\wedge$, AND), disjunction ($\vee$, OR), negation ($\neg$, NOT), implication ($\rightarrow$), and biconditional ($\leftrightarrow$). A **tautology** is a formula that is true under all assignments of truth values.

Predicate Logic (First-Order Logic) extends propositional logic with quantifiers. The **universal quantifier** $\forall x P(x)$ means "for all x , $P(x)$ is true." The **existential quantifier** $\exists x P(x)$ means "there exists an x such that $P(x)$ is true." Predicates are functions that return truth values.

7.2 Application to the System

The VAE Gate Decision

The central branching logic of the pipeline can be expressed as a propositional statement. Let:

- p = "the anomaly ratio exceeds 0.20" (i.e., $|P_anomalous| / |P| > 0.20$)

Then the pipeline's decision is:

```
p → classify(image)
¬p → returnNormal(image)
```

Equivalently: the system classifies the image if and only if p is true. This is a conditional statement: $p \rightarrow \text{classify}$. Its contrapositive is logically equivalent: $\neg \text{classify} \rightarrow \neg p$. The system implements this using a simple if-else branch in Dart, but the underlying structure is propositional logic.

Role-Based Navigation Logic

After authentication and role selection, the application routes the user based on their declared role. Let:

- r_p = "user role is Patient"
- r_s = "user role is Specialist"

These are mutually exclusive and exhaustive: $r_p \vee r_s = T$ (tautology, since every user must be one or the other), and $r_p \wedge r_s = F$ (contradiction, no user can be both). The routing logic is:

```
r_p → navigate(HomeScreen)
r_s → navigate(SpecialistDashboardScreen)
```

This is a formally complete decision procedure: for every possible user role, exactly one navigation target is defined.

Predicate Logic for Data Integrity

The database stores scan results. We can express data integrity constraints using predicate logic. Let $\text{ScanResult}(s)$ mean " s is a stored scan result," and let $\text{confidence}(s)$ denote its confidence value, and $\text{label}(s)$ its diagnosis label:

```

 $\forall s \ [ \text{ScanResult}(s) \rightarrow (0 \leq \text{confidence}(s) \leq 1) ]$ 
 $\forall s \ [ \text{ScanResult}(s) \rightarrow \text{label}(s) \in \{\text{Acne}, \text{Eczema}, \text{Tinea}, \text{Normal}\} ]$ 
 $\forall s \ [ \text{ScanResult}(s) \rightarrow \exists u \ [ \text{User}(u) \wedge \text{owns}(u, s) ] ]$ 

```

The third statement says every scan result must be owned by some user — expressed as an existential quantifier over users. These are the formal specifications of the database constraints, which in implementation are enforced by the Dart model classes and the SQLite schema.

Ensemble Correctness Proposition

A logical claim about the ensemble can be stated as follows:

```

"The ensemble's output confidence is always greater than or equal to the
minimum confidence of either individual model."

```

This is not always true in general, but it can be proven or disproven using the properties of the averaging function. If f_{B2} assigns probability 0.9 to Acne and f_{B3} assigns 0.5, the ensemble assigns 0.7. So: $\min(0.9, 0.5) = 0.5 \leq 0.7 = \text{ensemble}$. This holds in this case. Formally: for any x , $\text{ensemble}(x)_i = (f_{B2}(x)_i + f_{B3}(x)_i)/2 \geq \min(f_{B2}(x)_i, f_{B3}(x)_i)$, which is always true by the properties of averages.

Logical Completeness of Diagnosis

The system is logically complete — that is, every valid input produces exactly one output. This is expressed as:

```

 $\forall x \ [ \text{ValidImage}(x) \rightarrow \exists! d \ [ d \in D \wedge \text{diagnosis}(x) = d ] ]$ 

```

The symbol $\exists!$ means "there exists exactly one." The pipeline satisfies this because: the VAE gate either returns Normal (one output) or passes to the ensemble; the ensemble always produces a softmax vector; argmax always selects exactly one index; and the index maps to exactly one label in D_{disease} . The system is a total, well-defined function — a logical requirement for any reliable medical screening tool.

8. Concept V — Combinatorics

8.1 Formal Definition

Combinatorics is the branch of mathematics concerned with counting, arrangement, and combination of elements. Key principles include:

- **Multiplication Rule:** If task A can be done in m ways and task B in n ways, both together can be done in $m \times n$ ways.
- **Permutations:** The number of ordered arrangements of r elements from n is $P(n, r) = n! / (n-r)!$
- **Combinations:** The number of unordered selections of r elements from n is $C(n, r) = n! / (r! \times (n-r)!)$
- **Pigeonhole Principle:** If n items are placed into k containers and $n > k$, at least one container holds more than one item.

8.2 Application to the System

Counting Augmented Training Samples

Data augmentation artificially expands the training dataset by applying geometric and photometric transformations. The augmentation pipeline applies:

- Horizontal flip: 2 outcomes (flipped, not flipped)
- Rotation: angles sampled from $[-15^\circ, +15^\circ]$ — discretized to approximately 7 distinct values at 5° steps
- Zoom: factors from $[0.90, 1.10]$ — approximately 5 distinct values
- Brightness adjustment: factors from $[0.90, 1.10]$ — approximately 5 distinct values

By the **Multiplication Rule**, the number of distinct augmented versions of a single image is approximately:

$$2 \times 7 \times 5 \times 5 = 350 \text{ augmented versions per original image}$$

If the original dataset contains N images, the augmented training set can theoretically contain up to $350N$ distinct samples. In practice, augmentation is applied stochastically (samples are drawn from continuous distributions), so the effective diversity is even higher.

Ensemble Selection

Over the course of the twelve training phases, the following distinct model architectures were trained and evaluated: MobileNetV2, EfficientNetB0, EfficientNetB2 ($\times 2$ variants), EfficientNetB3, and EfficientNetB4 — approximately 6 distinct model checkpoints. The number of possible 2-model ensembles from 6 models is:

$$C(6, 2) = 6! / (2! \times 4!) = 15 \text{ possible pairs}$$

The number of possible 3-model ensembles is:

$$C(6, 3) = 6! / (3! \times 3!) = 20 \text{ possible triples}$$

The chosen ensemble (B2 + B3) is one of these 15 pairs. The selection was made empirically based on validation accuracy, but combinatorics quantifies the size of the search space that was (at least partially) explored.

Test-Time Augmentation Combinations

Test-time augmentation (TTA) applies 10 augmented transformations to the test image at inference time, averaging the predictions. The 10 augmentations are drawn from the same transformation space described above. The number of ways to choose 10 ordered augmentations from a space of approximately 350 possible transforms is:

$$P(350, 10) = 350! / 340! \approx 2.14 \times 10^{25}$$

This figure is computationally intractable. In practice, the TTA augmentations are sampled randomly from this space rather than enumerated exhaustively. The combinatorial analysis demonstrates that 10 randomly drawn samples provide meaningful diversity without exhaustive search — a principled justification for the chosen TTA count.

Counting Patches in the Sliding Window

For a representative smartphone photograph, consider an image of height $H = 1280$ and width $W = 960$ (after the maximum-dimension constraint). The sliding window produces:

```
Rows of patches: [(1280 - 64) / 32] + 1 = [1216 / 32] + 1 = 38 + 1 = 39
Columns of patches: [(960 - 64) / 32] + 1 = [896 / 32] + 1 = 28 + 1 = 29
Total patches: 39 × 29 = 1,131 patches
```

Each patch is a $64 \times 64 \times 3$ tensor. The VAE processes all 1,131 patches to produce the anomaly ratio. This direct application of the multiplication rule gives a precise count of the computational workload for the VAE stage.

Score-CAM Channel Selection

The EfficientNetB3 backbone produces n feature map channels. The number of ways to choose 30 channels from n is $C(n, 30)$. For EfficientNetB3's final convolutional block, where $n = 1536$:

$$C(1536, 30) = 1536! / (30! \times 1506!)$$

This is an astronomically large number, which explains why Score-CAM does not try all possible subsets but instead uses a greedy ranking by activation magnitude to select the top 30. This is a heuristic approximation of an otherwise intractable combinatorial problem.

9. Concept VI — Algorithms and Complexity

9.1 Formal Definition

An **algorithm** is a finite, deterministic sequence of instructions that solves a well-defined computational problem. The **time complexity** of an algorithm, expressed using Big-O notation, describes how the running time grows as a function of the input size. Common complexities include:

- $O(1)$: constant time
- $O(\log n)$: logarithmic time
- $O(n)$: linear time
- $O(n \log n)$: linearithmic time
- $O(n^2)$: quadratic time

A **greedy algorithm** makes locally optimal choices at each step. A **divide-and-conquer** algorithm splits the problem into smaller subproblems, solves them independently, and combines results.

9.2 Application to the System

The Sliding Window Algorithm (VAE Gate)

```

Algorithm:      SlidingWindowAnomalyDetection(image,      window=64,      stride=32,
threshold=0)
Input: RGB image of size H × W × 3
Output: Boolean isAnomalous, Float anomalyRatio

anomalyCount ← 0
totalPatches ← 0
for i from 0 to ⌊(H - window) / stride⌋:
    for j from 0 to ⌊(W - window) / stride⌋:
        patch ← image[i*stride : i*stride+window, j*stride : j*stride+window]
        reconstruction ← VAE.forward(patch)
        mse ← mean((patch - reconstruction)2)
        if mse > ̸:
            anomalyCount ← anomalyCount + 1
            totalPatches ← totalPatches + 1
anomalyRatio ← anomalyCount / totalPatches
return anomalyRatio > 0.20, anomalyRatio

```

The time complexity of this algorithm is $O(H \times W / \text{stride}^2)$ for the outer loops, with each iteration running in $O(1)$ relative to the image size (since the VAE operates on fixed-size 64×64 patches). For our 1280×960 example, this is $O(1,131) = O(1)$ relative to the number of patches, but $O(H \cdot W)$ relative to the pixel count with stride factored out.

The Argmax Classification Algorithm

```

Algorithm: ArgmaxClassify(probabilityVector)
Input: Float vector p of length 3
Output: Integer index ∈ {0, 1, 2}

maxVal ← p[0]
maxIdx ← 0
for i from 1 to 2:
    if p[i] > maxVal:
        maxVal ← p[i]
        maxIdx ← i
return maxIdx

```

This is $O(|D_{\text{disease}}|) = O(3) = O(1)$ — constant time. The argmax operation over a fixed-size output space is trivially efficient, but it is still an algorithm with formal correctness guarantees: it always terminates, always returns exactly one index, and returns the index of the maximum element.

Score-CAM Top-K Channel Selection

```

Algorithm: TopKChannelSelect(channels, k=30)
Input: Array of n channel activation maps with magnitudes  $m_1, m_2, \dots, m_n$ 
Output: Array of k channel indices sorted by magnitude (descending)

Sort channels by  $m_i$  in descending order     $\leftarrow O(n \log n)$ 
Return first k indices                       $\leftarrow O(k)$ 
Total:  $O(n \log n)$ 

```

The dominant cost is the sorting step, giving $O(n \log n)$ complexity. For $n = 1536$ (EfficientNetB3 channels): $1536 \times \log_2(1536) \approx 1536 \times 10.6 \approx 16,282$ operations. This is fast in practice and justifies using sorting rather than a more complex selection algorithm.

Full Inference Pipeline — Complexity Summary

Stage	Algorithm Type	Complexity
Preprocessing	Conditional resize (bilinear)	$O(H \times W)$
VAE Sliding Window	Nested loop + forward pass	$O(H \times W / \text{stride}^2)$
CNN B2 Inference	Fixed-size matrix operations	$O(1)$ relative to input
CNN B3 Inference	Fixed-size matrix operations	$O(1)$ relative to input
Ensemble Average	Element-wise vector average	$O(D) = O(3)$
Argmax	Linear scan	$O(D) = O(3)$
Score-CAM Channel Select	Sort + slice	$O(n \log n)$
Score-CAM Heatmap Sum	Weighted sum over k channels	$O(k \times H \times W)$
SQLite Store	B-tree insert	$O(\log m)$ where m = record count

The overall pipeline is dominated by the VAE sliding window and the Score-CAM heatmap sum. Both are polynomial (in fact, low-degree polynomial) in the image dimensions, which is what makes on-device real-time inference feasible.

The EarlyStopping Algorithm

During training, the EarlyStopping callback monitors validation loss and can be formally described as follows:

```

Algorithm: EarlyStopping(patience=10)
State: bestLoss  $\leftarrow \infty$ , waitCount  $\leftarrow 0$ 

on each epoch:
    if val_loss < bestLoss:
        bestLoss  $\leftarrow$  val_loss
        waitCount  $\leftarrow 0$ 
        saveModel()
    else:
        waitCount  $\leftarrow$  waitCount + 1
        if waitCount  $\geq$  patience:
            terminateTraining()
            return

```

This algorithm has a formal termination proof: either the loss improves indefinitely (bounded by zero, so it must eventually stop improving), or the patience counter reaches its limit and training stops. The algorithm terminates in at most (maxEpochs) iterations — it cannot run forever, which is a formal correctness property.

10. Mathematical Modeling: Unified Formal Description

With each Discrete Mathematics concept established in context, this section presents a unified formal model of the entire system, consolidating the definitions and functions introduced in the preceding sections.

Definition 1 (Input Space). Let $X = \mathbb{R}^{(H \times W \times 3)}$ be the space of RGB images with height H , width W , and 3 color channels.

Definition 2 (Output Space). Let $D = \{0, 1, 2, 3\}$ represent $\{\text{Acne}, \text{Eczema}, \text{Tinea}, \text{Normal}\}$ respectively. A scan result is a tuple $(d, c, p, h) \in D \times [0,1] \times [0,1]^3 \times \text{FilePath}$, where d is the diagnosis, c is confidence, p is the class probability vector, and h is the heatmap file path.

Definition 3 (Preprocessing). The preprocessing function $\pi: X \rightarrow X$ is defined as:

```

 $\pi(x) = \text{downsample}(x) \text{ if } \max(H, W) > 1280, \text{ else } x$ 

```

Definition 4 (Patch Extraction). Given a preprocessed image x of size $H \times W$, the patch extraction function $\phi: X \rightarrow P$ generates the set P of overlapping 64×64 patches as described in Section 4.

Definition 5 (VAE Anomaly Gate). The VAE defines a reconstruction function $r: \mathbb{R}^{(64 \times 64 \times 3)} \rightarrow \mathbb{R}^{(64 \times 64 \times 3)}$. The anomaly score of a patch p is $\text{MSE}(p, r(p))$. The gate function $g: X \rightarrow \{0, 1\}$ is:

$$g(x) = 1 \text{ if } |\{p \in \varphi(x) : \text{MSE}(p, r(p)) > \theta\}| / |\varphi(x)| > 0.20, \text{ else } 0$$

Definition 6 (CNN Ensemble). Let $f_B2: X \rightarrow [0,1]^3$ and $f_B3: X \rightarrow [0,1]^3$ be the softmax output functions of EfficientNetB2 and EfficientNetB3 respectively. The ensemble function is:

$$E(x) = (f_B2(\text{resize}_{260}(\pi(x))) + f_B3(\text{resize}_{300}(\pi(x)))) / 2$$

Definition 7 (Diagnosis Function). The diagnosis function $\delta: X \rightarrow D$ is:

$$\delta(x) = 3 \text{ (Normal) if } g(x) = 0, \text{ else } \text{argmax}(E(x))$$

Definition 8 (Score-CAM Heatmap). Let $\Phi: X \rightarrow \mathbb{R}^{(H' \times W' \times n)}$ be the feature extraction function of EfficientNetB3's final convolutional layer. Let $S \subseteq \{1, \dots, n\}$ with $|S| = 30$ be the top-K selected channels by activation magnitude. The heatmap function $\mathcal{H}: X \times \mathbb{Z} \rightarrow \mathbb{R}^{(H \times W)}$ is:

$$\mathcal{H}(x, d) = \sum_{i \in S} w(i) \cdot \text{upsample}(\Phi(x)[:, :, i]) \quad \text{where} \quad w(i) = \text{softmax}(\{\text{score}(x, d, j) : j \in S\})_i$$

Theorem (Pipeline Totality). For every valid input $x \in X$, the diagnosis function $\delta(x)$ produces exactly one element of D .

Proof. The preprocessing function π is total by definition. The patch extraction function φ produces at least one patch for any valid image. The gate function g maps to $\{0, 1\}$ and is therefore total. If $g(x) = 0$, then $\delta(x) = 3$ immediately. If $g(x) = 1$, then $E(x)$ is the average of two softmax output vectors, which is itself a valid probability vector; the argmax of a non-empty finite vector is always uniquely defined (ties are broken by index). Therefore δ is total. ■

11. Implementation Mapping

Table 1 maps each Discrete Mathematics concept to the specific file, function, or component in the codebase where it is concretely realized.

Concept	Formal Construct	Implementation Location
Set Theory	$D = \{\text{Acne, Eczema, Tinea, Normal}\}$	<code>Flutter/lib/models/scan_result.dart</code>
Set Theory	$P_{\text{anomalous}} \subseteq P$ (anomalous patch subset)	<code>Flutter/lib/services/ai/ai_service.dart</code> , VAE loop
Set Theory	$S \subseteq C, S = 30$ (top-K channels)	<code>Flutter/lib/services/ai/ai_service.dart</code> , Score-CAM section
Set Theory	$\text{Train} \cap \text{Val} = \emptyset$ (dataset partition)	<code>first_code_model_training.py</code> , <code>train_test_split</code> calls
Relations	$R_{\text{user_scan}} \subseteq \text{Users} \times \text{Scans}$ (one-to-many)	<code>Flutter/lib/services/database/database_service.dart</code>
Functions	$f: \text{ImageSpace} \rightarrow D \times [0,1]$ (classification)	<code>Flutter/lib/services/ai/ai_service.dart</code> , <code>analyze()</code>
Functions	$\text{ensemble}(x) = (f_{B2} + f_{B3}) / 2$	<code>Flutter/lib/services/ai/ai_service.dart</code> , ensemble block
Functions	$w(i) = \text{softmax}(\text{scores})$ (Score-CAM weights)	<code>Flutter/lib/services/ai/ai_service.dart</code> , Score-CAM weights
Graph Theory	Navigation DAG / directed graph	<code>Flutter/lib/core/routing/app_router.dart</code>
Graph Theory	Pipeline DAG (preprocess $\rightarrow$ VAE $\rightarrow$ CNN $\rightarrow$...)	<code>Flutter/lib/services/ai/ai_service.dart</code> , <code>analyze()</code> flow
Graph Theory	Training dependency graph	Phases 1–12: <code>first_code_*.py</code> through <code>twelfth_code_*.py</code>

Concept	Formal Construct	Implementation Location
Logic	$p \rightarrow \text{classify};$ $\neg p \rightarrow \text{normal}$ (VAE gate)	Flutter/lib/services/ai/ai_service.dart , if-else gate
Logic	$r_p \rightarrow \text{Home};$ $r_s \rightarrow$ Dashboard (role routing)	Flutter/lib/core/routing/app_router.dart , role check
Logic	$\forall s \quad [$ ScanResult(s) $\rightarrow \text{constraints}]$	Flutter/lib/models/scan_result.dart , Flutter/lib/models/user.dart
Combinatorics	$2 \times 7 \times 5 \times 5 = 350$ augmentations per image	first_code_model_training.py , ImageDataGenerator
Combinatorics	$C(6,2) = 15$ possible ensembles	eighth_code_ensemble_tta.py , design decision
Combinatorics	$39 \times 29 = 1,131$ patches (1280×960 image)	Computed from ai_service.dart sliding window params
Algorithms	Sliding window $O(H \times W / \text{stride}^2)$	Flutter/lib/services/ai/ai_service.dart , VAE loop
Algorithms	Argmax $O(D)$ $= O(3)$	Flutter/lib/services/ai/ai_service.dart , argmax call
Algorithms	Top-K sort $O(n \log n)$	Flutter/lib/services/ai/ai_service.dart , Score-CAM
Algorithms	EarlyStopping (termination guarantee)	first_code_model_training.py twelfth_code_*.py through

12. Diagram Placeholders

The following diagrams are recommended to accompany this document as visual aids. Each description is sufficiently detailed to serve as a drawing specification.

Figure 1 — System Architecture Overview

A three-layer block diagram. Top layer: "User (Patient / Specialist)." Middle layer: "Flutter Application," with internal sub-blocks for "UI Screens," "Riverpod State," and "SQLite Database." Bottom layer: "AI Inference Pipeline," with sequential sub-blocks "Preprocess → VAE Gate → CNN Ensemble (B2 + B3) → Argmax → Score-CAM." Arrows flow downward from User to App to Pipeline during inference, and upward from Pipeline to App to User when returning results.

Figure 2 — Inference Pipeline DAG with Conditional Branch

A directed acyclic graph. Rectangular nodes: Input Image, Preprocessing, VAE Sliding Window, CNN B2, CNN B3, Average Softmax, Argmax, Score-CAM, Store Result, Display. The VAE Sliding Window node is a diamond (decision node) with two outgoing edges: one labeled "anomaly ratio < 0.20" leading directly to a terminal Display node (Normal result); one labeled "anomaly ratio ≥ 0.20 " leading to the parallel CNN B2 and CNN B3 nodes, which converge at Average Softmax, then continue through the remaining pipeline.

Figure 3 — Application Navigation Directed Graph

A directed graph with 18 nodes representing application screens. The onboarding subgraph (left): a strictly linear chain with no back edges — Welcome → Login → RoleSelection → PersonalInfo → AllSet. The main-application subgraph (right): Home and SpecialistDashboard as high-degree hub nodes, with outgoing edges to BodyPartSelection, History, Profile, Notifications, and HelpSupport. The core workflow sub-chain: BodyPartSelection → Capture → Analyzing → DiagnosisResult, with DiagnosisResult having two outgoing edges (to History and to Home). Back-navigation edges from most screens to Home are drawn as curved dashed arcs to distinguish them from forward transitions.

Figure 4 — Sliding Window Patch Grid (VAE Input)

A 1280×960 pixel image with a 64×64 sliding window overlaid at stride 32. The top-left 4×4 region of patches is drawn explicitly; overlapping areas between adjacent patches are shown with light shading to visualize stride overlap. One patch is highlighted in red and labeled "anomalous: MSE > 0"; surrounding patches are green and labeled "normal." The total patch count ($39 \times 29 = 1,131$) is displayed below the figure.

Figure 5 — Score-CAM Heatmap Generation Pipeline

A horizontal flow diagram. Far left: the original skin photograph. An arrow labeled "EfficientNetB3 backbone" leads to a 3D cube representing $n = 1536$ feature map channels. An arrow labeled "Top-30 selection (by L2 norm)" leads to a smaller cube of 30 channels. Each of the 30 channels is shown

multiplied by its corresponding softmax weight w_i . A summation symbol collects the 30 weighted channels into a single grayscale heatmap. A final arrow labeled "Jet colormap overlay (60% original + 40% heatmap)" produces the colored explainability image on the far right.

Figure 6 — Set-Theoretic Visualization of Dataset Partition and Output Space

Upper half: a rectangle representing the universal set U of all labeled images. Inside U , three mutually disjoint ellipses labeled Train ($\sim 90\%$), Val ($\sim 5\%$), and Test ($\sim 5\%$) illustrate the dataset partition. A shaded sub-region within Train is labeled "Augmented Samples ($\in$ Train only)" to show that augmentation does not propagate to Val or Test. Lower half: a separate set diagram showing $D = \{\text{Acne, Eczema, Tinea, Normal}\}$ as the full output set, with $D_{\text{disease}} = \{\text{Acne, Eczema, Tinea}\}$ drawn as a proper subset ellipse inside D , and the singleton $\{\text{Normal}\}$ outside D_{disease} but inside D .

13. Conclusion

This paper has demonstrated, with formal rigor and concrete specificity, that Discrete Mathematics is not merely a theoretical prerequisite for computer science — it is the precise language in which real AI systems are designed and understood.

Set Theory provided the vocabulary for defining the output space of the classifier, formally specifying the dataset partition that prevents data leakage, and precisely characterizing the subsets of patches and feature channels that drive the VAE gate and Score-CAM heatmap.

Relations and Functions formalized the classification pipeline as a composition of total, well-defined functions, established the one-to-many user-scan relationship in the database, and gave a rigorous definition of the Score-CAM weighting as a probability mass function via the softmax.

Graph Theory revealed the application's navigation system as a directed graph, the AI pipeline as a DAG with a conditional branch, and the training process as a dependency graph — all structures with formal properties (acyclicity, connectivity, reachability) that directly impact the correctness and behavior of the system.

Propositional and Predicate Logic expressed the VAE gate as a conditional implication, role-based routing as a logically complete decision procedure, and database integrity as first-order predicate constraints. The formal proof of pipeline totality established that the system always produces exactly one diagnosis — a necessary property for any medical screening tool.

Combinatorics quantified the scale of data augmentation, the number of possible ensembles that were implicitly explored during training, the exact patch count for a given image resolution, and justified the greedy approximation used by Score-CAM in lieu of exhaustive channel enumeration.

Algorithm Analysis provided time complexity estimates for every stage of the inference pipeline, gave formal pseudocode descriptions of the sliding window, argmax, and Score-CAM algorithms, and proved the termination of the EarlyStopping training callback.

Together, these six branches of Discrete Mathematics form a complete formal model of the system described in Definition 7. The system is correct, total, and efficient — and each of these properties can be stated and verified using the concepts introduced in this paper.

14. References

- [1] Rosen, K. H. (2019). *Discrete Mathematics and Its Applications* (8th ed.). McGraw-Hill Education.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- [3] Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 6105–6114. PMLR.
- [4] Wang, H., Wang, Z., Du, M., Yang, F., Zhang, Z., Ding, S., Mardziel, P., & Hu, X. (2020). Score-CAM: Score-weighted visual explanations for convolutional neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 24–25.
- [5] Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988.
- [6] Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. In *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*.
- [7] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*.
- [8] Esselink, R. (2024). *Riverpod: A reactive caching and data-binding framework for Flutter and Dart* (Version 2.4.9) [Software library]. pub.dev.
- [9] Google LLC. (2024). *TensorFlow Lite: Machine learning for mobile and edge devices* (Version 2.x) [Software framework]. TensorFlow. <https://www.tensorflow.org/lite>
- [10] Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.